

Manual Técnico — Prototipo CHANDO

Documentación técnica del prototipo web del **sistema digital de transformación del canal de marketing de CHANDO Cosméticos**. Está dirigida a desarrolladores y revisores; para el uso funcional paso a paso, ver el [Manual de Usuario](#).

Índice

1. [Descripción general](#)
 2. [Arquitectura](#)
 3. [Stack y dependencias](#)
 4. [Requisitos, instalación y ejecución](#)
 5. [Estructura de carpetas](#)
 6. [Enrutamiento](#)
 7. [Gestión de estado \(Context API\)](#)
 8. [Modelo de datos \(JSON\)](#)
 9. [Componentes reutilizables](#)
 10. [Módulos / páginas](#)
 11. [Lógica de negocio clave](#)
 12. [Estilos y tema](#)
 13. [Notificaciones \(react-hot-toast\)](#)
 14. [Generación de capturas \(Playwright\)](#)
 15. [Build y despliegue](#)
 16. [Limitaciones y mejoras futuras](#)
-

1. Descripción general

- **Tipo:** SPA (Single Page Application) 100 % front-end. **No hay backend ni base de datos.**
 - **Datos:** archivos JSON en `src/data/` que se cargan a memoria. El estado mutable (inventario, pedidos, alertas, carrito) vive en React mientras la app está abierta y se reinicia al recargar la página.
 - **Autenticación:** simulada (selección de rol, sin contraseñas ni persistencia).
 - **Caso simulado:** consolidación logística de 177 → 14 bodegas, despacho de 1 semana → 2 horas, entrega de 1 mes → 45 horas (valores usados en el módulo ejecutivo).
-

2. Arquitectura

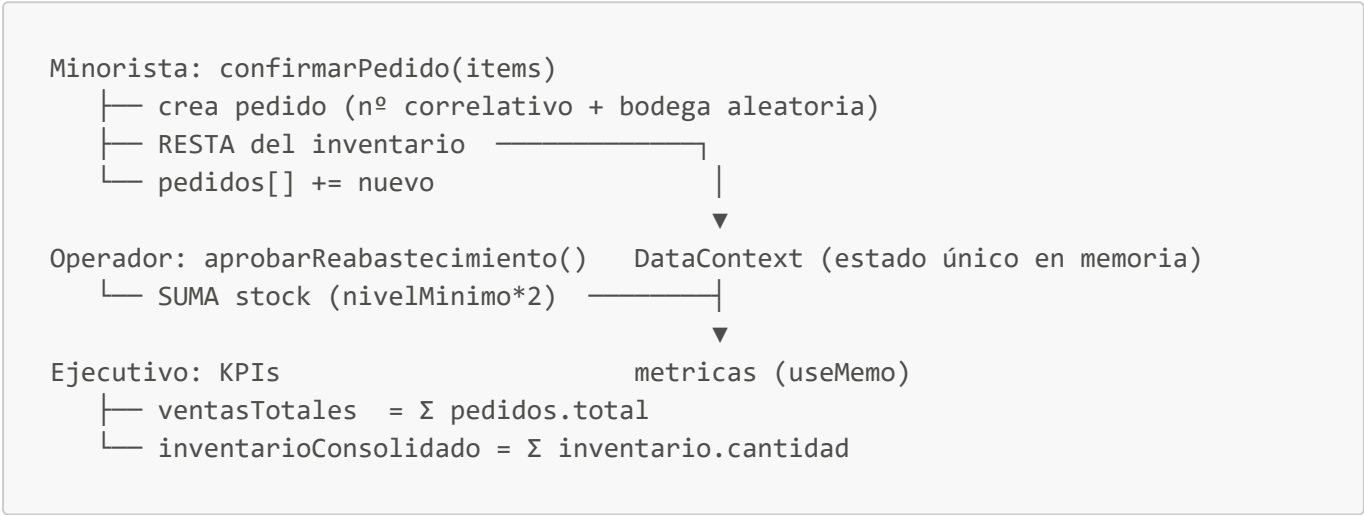
Aplicación React renderizada en cliente, con enrutamiento por React Router y estado global mediante Context API. Los tres módulos comparten el mismo estado de dominio (`DataContext`), por lo que las acciones de un rol impactan a los demás.

Árbol de proveedores (`src/main.jsx`)

```
<BrowserRouter>
  └─ <AuthProvider>           // rol de la sesión
    └─ <DataProvider>         // inventario, pedidos, alertas, KPIs (estado
compartido)
      └─ <App/>               // rutas
      └─ <Toaster/>           // notificaciones react-hot-toast
```

CartProvider no es global: solo envuelve al **RetailerPortal** (el carrito es exclusivo del minorista).

Flujo de datos conectados



Layout de cada módulo

```
<div h-screen flex-col>
  <Header/> // barra superior: logo · título · usuario ·
logout
  <div flex>
    <Sidebar/> // navegación entre vistas (tabs con useState)
    <main overflow-y-auto> // contenido de la vista activa (animate-fade-in)
  </div>
</div>
```

3. Stack y dependencias

Producción

Paquete	Versión	Uso
react / react-dom	^18.3.1	Librería de UI
react-router-dom	^6.30.3	Enrutamiento SPA
recharts	^3.8.1	Gráficos del dashboard ejecutivo

Paquete	Versión	Uso
lucide-react	^1.16.0	Iconografía
react-hot-toast	^2.6.0	Notificaciones (toasts)

Desarrollo

Paquete	Versión	Uso
vite	^5.4.21	Bundler y dev server
@vitejs/plugin-react	^4.7.0	Soporte React/JSX/HMR
tailwindcss	^3.4.19	Framework de estilos
postcss	^8.5.15	Procesador CSS (requerido por Tailwind)
autoprefixer	^10.5.0	Prefijos CSS automáticos

`playwright` se usa **solo** para generar capturas y se instala con `--no-save` (no figura en `package.json`).

4. Requisitos, instalación y ejecución

Requisito: **Node.js 18+** y npm.

```
# Instalar dependencias (primera vez)
npm install

# Servidor de desarrollo con HMR -> http://localhost:5173
npm run dev

# Build de producción -> carpeta /dist
npm run build

# Previsualizar el build de producción
npm run preview
```

Scripts definidos en `package.json`:

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview"
}
```

5. Estructura de carpetas

PROTOTIPO/	
├─ public/	
│ └─ chando-icon.svg	Favicon
├─ src/	
│ └─ components/	Componentes reutilizables
│ │ └─ Sidebar.jsx	Barra lateral (items + active + onSelect + rol)
│ │ └─ Header.jsx	Encabezado (logo, título, usuario, logout,
│ │ acciones)	
│ │ └─ Card.jsx	Card genérica + StatCard (export nombrado)
│ │ └─ Button.jsx	Botón con variantes/animaciones
│ └─ pages/	
│ │ └─ Login.jsx	Selección de rol
│ │ └─ RetailerPortal.jsx	Portal del Minorista (3 vistas)
│ │ └─ InventoryCenter.jsx	Centro de Control de Inventario (3 vistas)
│ │ └─ ExecutiveDashboard.jsx	Dashboard Ejecutivo (3 vistas, Recharts)
│ └─ context/	
│ │ └─ AuthContext.jsx	Rol del usuario (login/logout)
│ │ └─ CartContext.jsx	Carrito del minorista (UI)
│ │ └─ DataContext.jsx	Inventario/pedidos/alertas + KPIs (estado
│ │ compartido)	
│ │ └─ data/	Datos simulados (JSON)
│ │ │ └─ productos.json	bodegas.json inventario.json
│ │ │ └─ pedidos.json	kpis.json campaigns.json
│ │ └─ assets/	
│ │ │ └─ chando-logo.svg	Logo monograma
│ │ └─ App.jsx	Definición de rutas
│ │ └─ main.jsx	Punto de entrada + proveedores + Toaster
│ │ └─ index.css	Directivas Tailwind + estilos globales
├─ docs/img/	Capturas usadas por los manuales
├─ scripts/take-screenshots.mjs	Script Playwright para regenerar capturas
├─ tailwind.config.js	Paleta corporativa + animaciones
├─ postcss.config.js	
├─ vite.config.js	
├─ package.json	
├─ README.md	Documentación técnica resumida
├─ MANUAL.md	Manual de usuario (con capturas)
└─ MANUAL-TECNICO.md	Este documento

6. Enrutamiento

Definido en `src/App.jsx` con React Router v6:

Ruta	Componente	Rol
/	Login	—
/minorista	RetailerPortal	minorista
/inventario	InventoryCenter	operador
/ejecutivo	ExecutiveDashboard	ejecutivo

```
<Routes>
  <Route path="/" element={<Login />} />
  <Route path="/minorista" element={<RetailerPortal />} />
  <Route path="/inventario" element={<InventoryCenter />} />
  <Route path="/ejecutivo" element={<ExecutiveDashboard />} />
</Routes>
```

No hay guardas de ruta: las páginas internas se renderizan aunque se acceda directo por URL (apropiado para un prototipo). La navegación entre módulos se hace vía *Cerrar sesión* → *Login* → *elegir rol* (navegación SPA, conserva el estado de `DataContext`).

7. Gestión de estado (Context API)

7.1 AuthContext (`src/context/AuthContext.jsx`)

```
usuario          // null | { rol }
login(rol)       // setUsuario({ rol })
logout()         // setUsuario(null)
```

Constantes exportadas:

```
ROLE_ROUTES = { minorista: '/minorista', operador: '/inventario', ejecutivo:
'/ejecutivo' }
ROLE_LABELS = { minorista: 'Minorista', operador: 'Operador de Inventario',
ejecutivo: 'Ejecutivo CHANDO' }
```

7.2 CartContext (`src/context/CartContext.jsx`)

Solo estado de UI del carrito; **no** crea pedidos.

```
cart          // { [productoId]: cantidad }
items         // [{ ...producto, cantidad, subtotal }] (derivado,
useMemo)
totalItems, subtotal // totales derivados
agregar(id)
cambiarCantidad(id, delta) // si llega a 0, elimina el ítem
quitar(id)
vaciar()
```

7.3 DataContext (`src/context/DataContext.jsx`)

Estado de dominio **compartido** por los tres módulos. Inicializa desde los JSON y muta en memoria.

Estado interno:

```

inventario      // copia de inventario.json (mutable)
pedidos         // copia de pedidos.json (mutable)
estadoAlertas   // { 'productoId-bodegaId': 'procesada' | 'rechazada' }

```

Confirmar pedido (resta inventario):

```

const confirmarPedido = (items, minoristaId = 'MIN-014') => {
  if (!items?.length) return null
  const total = items.reduce((s, i) => s + i.precio * i.cantidad, 0)
  const maxNum = pedidos.reduce((m, p) => Math.max(m,
    Number(String(p.numeroPedido).split('-').pop()) || 0), 0)
  const numeroPedido = `PED-2026-${String(maxNum + 1).padStart(4, '0')}`
  const bodega = bodegas[Math.floor(Math.random() * bodegas.length)]
  const nuevo = { id: Date.now(), numeroPedido, minoristaId, fecha: HOY,
    productos: items.map(i => ({ productoId: i.id, cantidad: i.cantidad })),
    estado: 'confirmado', bodegaAsignada: bodega.id, total }
  setInventario(prev => restarInventario(prev, items, bodega.id)) // <--
  descuento stock
  setPedidos(prev => [nuevo, ...prev])
  return { ...nuevo, bodegaNombre: bodega.nombre }
}

```

`restarInventario` descuenta primero de la bodega asignada y luego de las entradas con más stock, sin permitir negativos.

Aprobar / rechazar reabastecimiento:

```

const aprobarReabastecimiento = (productoId, bodegaId) => {
  setInventario(prev => prev.map(e =>
    e.productoId === productoId && e.bodegaId === bodegaId
      ? { ...e, cantidad: e.cantidad + e.nivelMinimo * 2 } // <-- suma stock
      : e))
  setEstadoAlertas(prev => ({ ...prev, [`${productoId}-${bodegaId}`]: 'procesada'
  })))
}
const rechazarAlerta = (productoId, bodegaId) =>
  setEstadoAlertas(prev => ({ ...prev, [`${productoId}-${bodegaId}`]: 'rechazada'
  })))

```

Derivados (useMemo):

```

alertas // inventario.filter(cantidad < nivelMinimo) con estado y orden por
gravedad
metricas = {

```

```
totalSKUs, totalUnidades,          // Σ inventario.cantidad
bodegasActivas, totalBodegas,
alertasPendientes, procesadasHoy,
ventasTotales,                    // Σ pedidos.total
totalPedidos,
}
```

8. Modelo de datos (JSON)

productos.json (12 productos)

```
{ "id": 1, "sku": "CH-SK-001", "nombre": "Sérum Vitamina C Iluminador",
  "categoria": "skincare", "precio": 210,
  "imagen": "https://placeholder.co/300x300/E91E63/white?text=Serum+Vit+C",
  "stockTotal": 12400 }
```

categoria ∈ skincare | makeup | fragrance.

bodegas.json (14 bodegas)

```
{ "id": 1, "nombre": "Bodega Shanghái Central", "ciudad": "Shanghái",
  "capacidad": 80000, "ocupacionActual": 82, "estado": "activa" }
```

estado ∈ activa | mantenimiento.

inventario.json (relación producto × bodega, 71 registros)

```
{ "productoId": 1, "bodegaId": 1, "cantidad": 1800,
  "nivelMinimo": 500, "ultimaActualizacion": "2026-05-22" }
```

pedidos.json (8 pedidos)

```
{ "id": 1, "numeroPedido": "PED-2026-0001", "minoristaId": "MIN-014",
  "fecha": "2026-05-02",
  "productos": [{ "productoId": 1, "cantidad": 20 }],
  "estado": "entregado", "bodegaAsignada": 1, "total": 7760 }
```

estado ∈ confirmado | preparacion | transito | entregado.

kpis.json

```
{
  "ventasTotalesMes": { "valor": "CNY 530M", "delta": "50.8%", "trend": "up",
    "positivo": true },
  "inventarioConsolidado": { "valor": "1.2M uds", "delta": "31%", "trend":
    "down", "positivo": true },
  "tiempoPromedioDespacho": { "valor": "2 horas", "delta": "desde 1 semana",
    "trend": "down", "positivo": true },
  "rotacionInventario": { "valor": "41% más rápida", "delta": "vs. proceso
    manual", "trend": "up", "positivo": true },
  "ventasPorMes": [{ "mes": "Dic", "ventas": 350 }, ...],
  "ventasPorRegion": [{ "region": "China Oriental", "ventas": 185 }, ...],
  "topProductos": [{ "nombre": "...", "sku": "CH-MK-001", "ventas": 18200 },
    ...],
  "comparativoAntesDespues": [
    { "metrica": "Procesamiento de pedidos", "antesReal": "1 semana",
      "despuesReal": "2 horas",
      "antesIdx": 100, "despuesIdx": 1, "mejora": "98.8%" }, ...
  ]
}
```

En el dashboard, **Ventas Totales del Mes** e **Inventario Consolidado** se **calculan** desde `DataContext` (no usan `kpis.json`); `tiempoPromedioDespacho` y `rotacionInventario` sí provienen de `kpis.json` porque no son derivables de las transacciones simuladas.

campaigns.json

Campañas de marketing. **Sin uso actual** (queda como dato de ejemplo para una futura vista).

9. Componentes reutilizables

Sidebar.jsx

```
<Sidebar
  rol="Minorista" // etiqueta del módulo (texto)
  items={[{ id, label, icon, badge?, badgeTone? }]}
  active={vista} // id del item activo
  onSelect={setVista} // (id) => void
/>
```

- `icon` es un componente de `lucide-react`.
- `badge` (número) y `badgeTone` (`'secondary'` | `'danger'`) muestran un contador (carrito, alertas).
- Responsive: riel de iconos (`w-16/20`) en móvil, completo (`md:w-60`) en escritorio.

Header.jsx

```
<Header
  titulo="Portal del Minorista" // string o JSX (centrado)
  usuario={{ nombre, sub }}    // bloque derecho
  acciones={<RangoSelector/>} // nodo opcional (ej. selector de fechas)
/>
```

Incluye el logo (izquierda, alineado con el ancho del sidebar) y el botón de **logout** (usa `useAuth().logout()` + `navigate('/')`).

Card.jsx

- **Card** — contenedor con borde, sombra suave y esquinas redondeadas.
- **StatCard**(`{ label, value, delta, positive, icon }`) — tarjeta de métrica con flecha de tendencia (verde si `positive`, roja si no).

Button.jsx

```
<Button variant="primary" size="md">...</Button>
```

- `variant`: `primary` | `secondary` | `outline` | `ghost` | `dark`.
- `size`: `sm` | `md` | `lg`.
- Animaciones: `hover:scale-[1.02]`, `active:scale-95`, `transition-all`.

10. Módulos / páginas

Todos siguen el mismo patrón: `useState` para la vista activa, `<Header/>` + `<Sidebar/>` compartidos, y un contenedor `animate-fade-in` con `key={vista}` para reanimar al cambiar.

Login.jsx

Selección de rol (3 tarjetas). Al pulsar *Ingresar*: `login(rol)` + `navigate(ROLE_ROUTES[rol])`.

RetailerPortal.jsx (envuelto en CartProvider)

- **Catálogo** — filtros (búsqueda + categoría con `useMemo`), badges de stock (`Sin stock` si `stockTotal === 0`, `Stock limitado` si `< 50`).
- **Carrito** — controles `+/-`, resumen, *Confirmar Pedido* → `useData().confirmarPedido(items)`
 - `vaciar()` + modal de éxito + toast.
- **Mis Pedidos** — lista de `useData().pedidos`, filtros por estado, tarjeta expandible con línea de tiempo y modal de detalle.

InventoryCenter.jsx

- **Vista General** — métricas de `useData().metrics` + grid de 14 bodegas con barra de ocupación (semáforo).

- **Detalle por Bodega** — `useData().inventario` filtrado por bodega; estado `OK/Bajo/Crítico`; filas críticas en rojo con botón *Reabastecer* (toast de solicitud).
- **Alertas** — `useData().alertas` pendientes; *Aprobar* (`aprobarReabastecimiento`) / *Rechazar* (`rechazarAlerta`) + toasts.

ExecutiveDashboard.jsx (Recharts)

- **KPIs** — 4 tarjetas (2 calculadas + 2 de transformación), `LineChart` (ventas por mes), `BarChart` (ventas por región), mapa de bodegas y top 5. Selector de rango en el header.
- **Análisis Comparativo** — tarjetas de mejora + `BarChart` agrupado (Antes/Después) con tooltip personalizado.
- **Reportes** — formulario (tipo, fechas, formato) → vista previa (`LineChart/BarChart` + tabla) según el tipo; *Generar Reporte* y *Descargar* disparan toasts.

11. Lógica de negocio clave

Regla	Definición
Estado de stock	<code>critico</code> si <code>cantidad < nivelMinimo*0.5</code> ; <code>bajo</code> si <code>cantidad < nivelMinimo</code> ; si no, <code>ok</code> .
Color de ocupación	<code>>90</code> rojo · <code>>=70</code> amarillo · <code><70</code> verde.
Badge catálogo	<code>stockTotal === 0</code> → "Sin stock" (botón deshabilitado); <code>< 50</code> → "Stock limitado".
Nº de pedido	<code>PED-2026-####</code> correlativo (máximo existente + 1).
Bodega asignada	Aleatoria entre las 14 al confirmar el pedido.
Cantidad sugerida	<code>nivelMinimo * 2</code> .
Reabastecer (aprobar)	Suma <code>nivelMinimo * 2</code> a <code>cantidad</code> .
KPI Ventas	Σ <code>pedidos.total</code> (calculado).
KPI Inventario	Σ <code>inventario.cantidad</code> (calculado).

12. Estilos y tema

Tailwind CSS v3. Paleta y animaciones en `tailwind.config.js`:

Token	Hex
<code>primary</code> / <code>.light</code> / <code>.dark</code>	<code>#E91E63</code> / <code>#F8BBD0</code> / <code>#C2185B</code>
<code>secondary</code> / <code>.light</code>	<code>#D4AF37</code> / <code>#E8D48A</code>
<code>dark</code>	<code>#1F2937</code>
<code>light</code>	<code>#FFF8F3</code>

```
// animaciones
keyframes: { 'fade-in': { '0%': { opacity:0, transform:'translateY(10px)' },
'100%': {...} } }
animation: { 'fade-in': 'fade-in 0.4s ease-out both', 'fade-in-fast': '... 0.25s
...' }
boxShadow: { soft: '0 4px 20px -4px rgba(233,30,99,0.12)' }
```

`src/index.css` incluye las directivas `@tailwind` y estilos globales (fuente Inter, scrollbar con color de marca).

13. Notificaciones (react-hot-toast)

`<Toaster/>` se monta una sola vez en `main.jsx` (posición `top-right`, duración 2800 ms, estilo oscuro).

Toasts disparados:

Mensaje	Origen
Producto agregado al carrito	Catálogo (agregar)
Pedido confirmado #PED-2026-####	Carrito (confirmar)
Solicitud de reabastecimiento enviada: ...	Detalle por bodega (reabastecer)
Reabastecimiento aprobado / Alerta rechazada	Alertas
Reporte generado exitosamente / Reporte descargado en formato ...	Reportes

14. Generación de capturas (Playwright)

Las imágenes de los manuales se generan automáticamente.

```
# 1) tener el dev server activo
npm run dev

# 2) (solo una vez) instalar Playwright + Chromium sin tocar package.json
npm install --no-save playwright
npx playwright install chromium

# 3) generar las capturas en docs/img/
node scripts/take-screenshots.mjs
```

El script (`scripts/take-screenshots.mjs`) abre Chromium headless, recorre las rutas, interactúa (login, agregar al carrito, confirmar, aprobar, generar reporte) y guarda 11 PNG a 1280 px de ancho.

15. Build y despliegue

```
npm run build      # genera /dist (HTML + JS + CSS minificados)
npm run preview    # sirve /dist localmente para validar
```

- El resultado en `/dist` es **estático**: se puede publicar en cualquier hosting estático (Netlify, Vercel, GitHub Pages, etc.). Por ser SPA, configurar *fallback* a `index.html`.
- Vite emite un aviso de *chunk > 500 kB*: es esperado (Recharts es pesado) y no afecta el funcionamiento. Si se quisiera optimizar: `build.rollupOptions.output.manualChunks` o `import()` dinámico.

16. Limitaciones y mejoras futuras

- **Sin persistencia**: el estado se reinicia al recargar (no hay `localStorage` ni backend).
- **Autenticación simulada**: sin credenciales ni control de acceso real por ruta.
- **KPIs mixtos**: despacho y rotación son valores narrativos del caso (en `kpis.json`), no calculados.
- **campaigns.json sin uso**: disponible para una futura vista de campañas.
- **Posibles extensiones**: persistir en `localStorage`, backend/API real, exportación real de PDF/Excel, pruebas automatizadas (Vitest/Playwright) y *code-splitting* por ruta.

Proyecto académico — Análisis de Sistemas 1, UMG. © 2026 CHANDO Cosméticos (ejemplo ficticio).